WATERLOO ROCKETRY

Next-Gen Ground Support Software System (Orion)

Preliminary Design Review

Qian Qian
Software Lead

Shrey Shingala
Software Lead

October 29, 2026

Agenda

01 Overview, Motivation & Feasibility

02 Requirements

03 Explanation of Key Technological Choices

04 Architecture Overview

05 System Architecture

06 Deployment Architecture

07 Message, Data, and API Definitions

08 Testing

09 Project Management

10 Action Items

SECTION 01

Overview, Motivation & Feasibility

01

WATERLOO ROCKETRY

Overview

Orion: overhauling Omnibus into its better successor.

- Same role as Omnibus: the hub between ground hardware (rocket CAN, LabJack DAQ, RLCS) and the operator applications.
- Rebuilt around one typed, versioned API, so sources and applications can change without breaking each other.
- A single Go server owns shared state, component health, command tracking and logging.
- Launch App (replacing the Omnibus Dashboard and DAQms) and Data Studio (replacing the data processing scripts) build on top of it.
- Staged migration: Omnibus stays production-ready until Orion fully replaces it.

Motivation

- Source-level breaking changes currently propagate across the stack, often requiring coordinated changes to dashboards and other consumers.
- Data contracts and system state are spread across multiple components, making behaviour harder to reason about and changes harder to validate.
- Adding new data sources and tooling increases maintenance overhead under the current architecture.
- We want a simpler, more reliable stack with stronger guarantees around shared data structures and interfaces.

Feasibility

- **Performance:** the expected load (CAN at hundreds of messages per second; DAQ at about 1 kHz across about 20 channels, batched) is well within the capacity of a Go server on a Raspberry Pi.
- **Technical maturity:** Go, Protobuf / gRPC and Electron / React are mature, widely adopted technologies, with code generation for every language in our stack.
- **Reuse of existing work:** the Parsley libraries, the LabJack integration and the Ansible / Docker deployment (`daq-raspi-deploy`) carry forward, reducing new development.
- **Incremental migration:** the Omnibus Adapter (#61) lets existing sources and consumers run alongside Orion, so migration proceeds component by component instead of as a single cut-over.
- **Risk management:** Omnibus remains the production fallback, scope is staged (MVP: the USB debug path end to end), and work is split into issues sized for new members.

SECTION 02

Requirements

02

WATERLOO ROCKETRY

Requirements Overview

The GitHub issues `2026-2027-software-issues#51` contains and is the root parent of issues containing:

- general technical directions
- design (tasks and decisions)
  - specification
  - API Protocol and Data Model
- implementation (requirements and tasks)

Please review its content.

General Requirements

Requirement	Motivation
Omnibus MUST remain production-ready until Orion is fully production-ready and is fully replacing Omnibus.	Allow smooth transition and ensure fallback option exists while issues are ironed out.
Orion SHOULD provide one authoritative API server for telemetry, shared state and command routing.	Provides a stable boundary between sources and consumers, and avoid system-wide breaks when a source or consumer is updated.
Public interfaces SHOULD be typed, versioned, documented and machine-checked.	Standardized interfaces to ensure safe interaction and validation of data.
Complete and well organized documentation MUST be provided for all public interfaces, including API, data model, and configuration.	Ensure that the system is usable and mainytainable by future developers and users.
The deployment of the system MUST be reproducible.	Ensure that the system can be deployed in a consistent manner.
The system MUST be easy to configure and the configuration methods MUST be documented.	Ensure that the system can be configured by all users. (Abandoning the current Python-script-based configuration system.)

SECTION 03

Explanation of Key Technological Choices

03

WATERLOO ROCKETRY

Go Stack

Replacing the current Python stack with a Go stack.

- It is a **typed** language, aligning with our goal of having typed messages schemas and interfaces.
- It compiles to a binary, which
  - is less dependent on the runtime environment,
  - is easier to deploy, and
  - is easier to distribute.

Go Stack - Key Changes

- Central Processing Server: switching from Python to Go
- Sink and Source modules: switching from Python to Go
- Dependency Management: switching from `uv` to Go's own dependency management tool.
- Testing Framework: switching from `pytest` to Go's built-in testing framework.
  - We might consider using other more feature-rich testing frameworks if it's deemed necessary during development.
- Networking: switching from WebSockets to Protobuf messages over gRPC,
  - it allows for typed messages, and
  - it supports both unary (request-response) and streaming operations, which is useful for supporting both command/info routing and telemetry data streaming.

Deployment

We continue to deploy on Raspberry Pi.

- Raspberry Pi OS, Docker, and Ansible are kept as they allow the deployment to be reproducible and can be used to redeploy the system quickly and consistently.
- Docker container may be further simplified compared to the Python stack, as the new stack is less dependent on the runtime environment and has fewer dependencies.
- Deliver a single binary, and use different CLI flags to configure the binary to run as different modules (e.g. server, sink, source, etc.), which is easier to deploy and maintain.

Launch App

Launch App will be a desktop app for use during rocket launches, replacing the old Omnibus Dashboard / DAQms. It will use **Electron** with **React and TypeScript**.

- Functional modern web development stack with easy compatibility on any operator's device.
- Easy to build, easy to deploy, easily compatible with the rest of our software.
- Orion mode uses decoded readings and semantic commands. Parsley-TS belongs to the direct USB integration.
- React is a well-known and well-supported frontend framework and is widely-used.

Data Studio

We are building Data Studio as a intended replacement for the current Omnibus “Data Processing” scripts, which are not well maintained and not intuitive to use.

Goal: a graphical app to review logs and export data, without needing to figure out how to use command-line scripts.

- **Import** Omnibus, Orion and other agreed log formats.
  - For Orion, it strictly follows the Orion data model and API.
- **Plot & Select** series and time ranges interactively.
- **Export** the selection as CSV, images, or other formats.

Data Studio

We want to use a modern web stack (e.g. React + TypeScript + D3.js) for Data Studio frontend, and use Go for the logic and data processing backend, which stays consistent with the rest of Orion system. `Wails` is the proposed framework for this stack.

- We might consider Vue for the frontend.
- Wails is sort of Electron, but it uses Go as the backend.
  - It is considered usable for the forseeable future and is actively maintained.
  - The newest version (v3.0.0) just released October 26th, 2026.
    The previous version (v2.17.0) was released mid-October 2026.
  - It was actively maintained with regular commits.
  - Unfortunately, it is not as popular and no public record shows that well-known companies or projects are using or backing it.

A dedicated PDR for Data Studio may be planned for the near future.

Lessons from Omnibus

Omnibus became hard to change because device knowledge leaked everywhere:

- Messages are a channel string plus an untyped dict, so every app hand-parses every source (`dashboard/parsers.py`, `omnibus-ts`, `can_field_definitions.py`).
- Apps import Parsley directly, so a RocketCAN change can break the dashboard.
- No registry or health model; commands are fire-and-forget raw CAN.
- Two transports (ZMQ and a SocketIO bridge) and Python-script configuration.

Orion's rule: device knowledge stays inside components.

Design Rules

Rule	Why
Decode at the component boundary: only the CAN Bridge (and the Launch App's direct USB mode) understand CAN.	Wire-format changes stay local; genuinely new public capabilities need review.
Generic observations and commands; source identity and capabilities are catalog data.	Adding a board or changing wire encoding stays local when the published meaning is unchanged.
Orion-mode apps use channel IDs, targets and operations. Parsley, CAN IDs and frame settings stay inside the integration component.	CAN and DAQ remain separate integrations; the shared API carries their decoded meanings.
Typed, versioned and machine-checked at every boundary.	Breaking changes are caught by fixtures (#86), not at launch.

SECTION 04

Architecture Overview

04

WATERLOO ROCKETRY

Architecture Overview

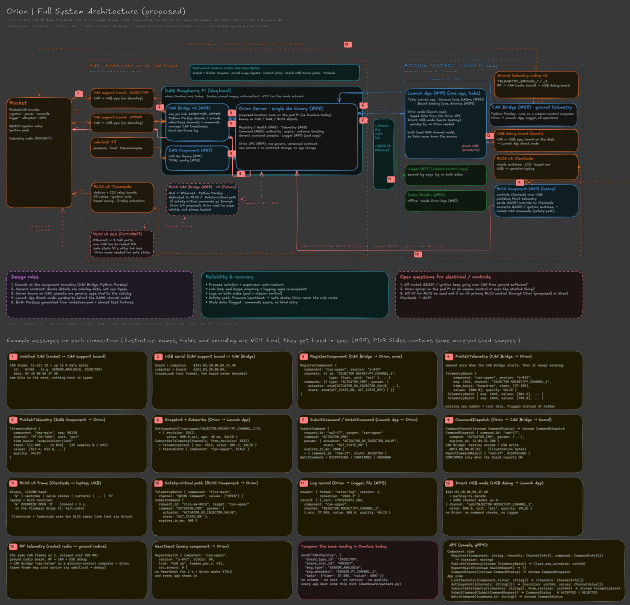


Diagram available as `assets/diagrams/orion-architecture-full.png` and Excalidraw drawing.

WATERLOO ROCKETRY

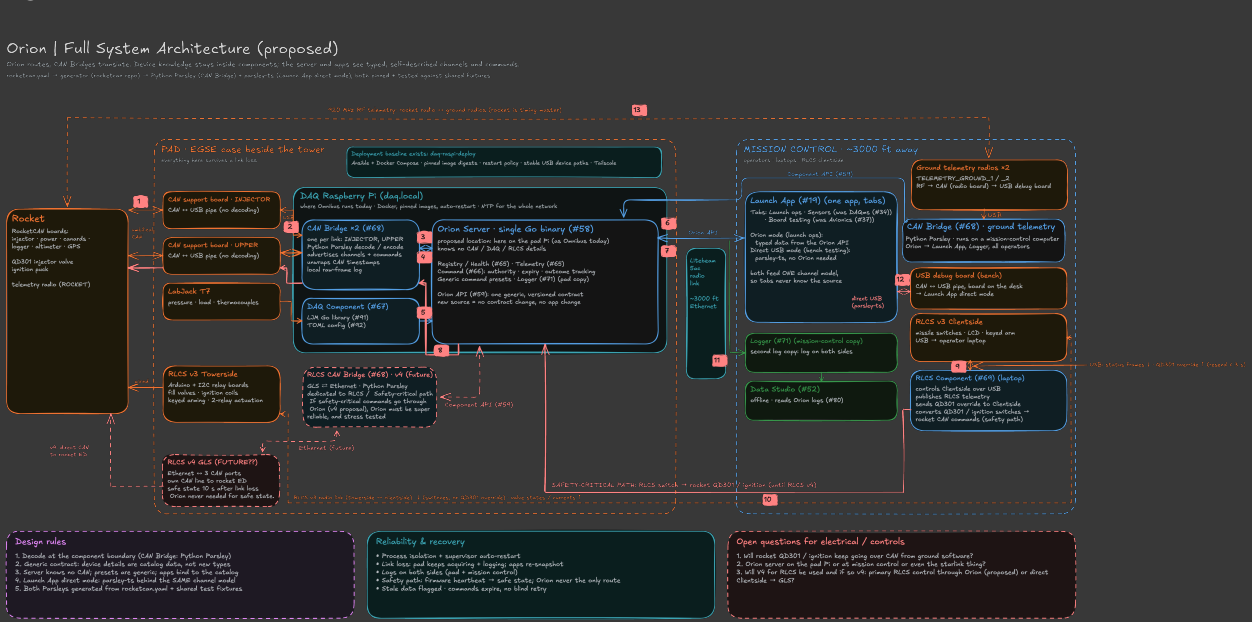
SECTION 05

System Architecture

05

WATERLOO ROCKETRY

System Architecture



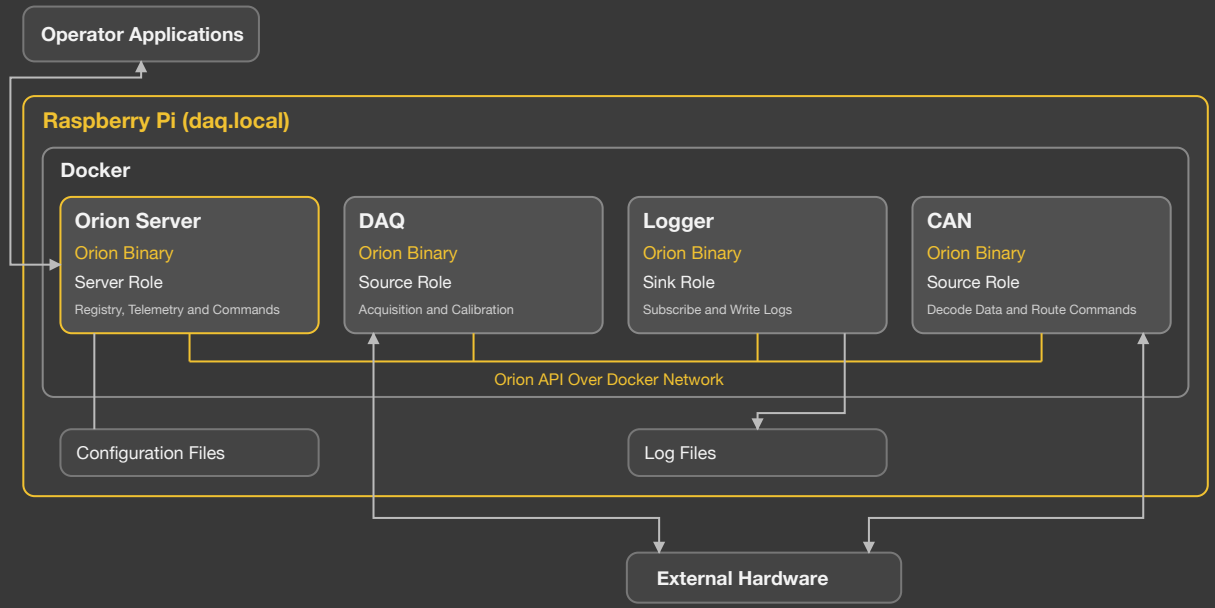
SECTION 06

Deployment Architecture

06

WATERLOO ROCKETRY

Raspberry Pi Deployment



Reliability and Recovery

- **Process isolation:** one crash never takes down another process; a supervisor restarts each one automatically.
- **Fast recovery:** after a server restart, components reconnect and re-register and apps re-snapshot, within seconds.
- **No silent data loss:** components buffer and log locally while the server is down; gaps are explicit.
- **Honest data:** stale or offline channels are always flagged, never shown as live.
- **Safe commands:** commands expire, are never retried blindly, and an ambiguous outcome is `UNKNOWN`.
- **Safety-critical path:** RLCS tower actuation is hardware. Rocket QD301 / ignition commands pass through Orion until RLCS v4, so that path gets stricter rules and firmware fail-safes.
- **Fallback:** Omnibus stays production-ready until Orion fully replaces it.
- **Deployment:** offline, reproducible, with tested rollback (#62).

SECTION 07

Message, Data, and API Definitions

07

WATERLOO ROCKETRY

Sample Messages and API Definitions

Example messages on each connection (illustrative: names, fields and encoding are NOT final; they get fixed in spec (#59), PDR Slides contains some more-polished samples.)

1 Umbilical CAN (rocket ↔ CAN support board)

```
CAN frame: 11-bit ID + up to 8 data bytes
  id:   0x2A1   (e.g. SENSOR_ANALOG16, INJECTOR)
  data: 03 20 00 0A 1F 40
raw bits on the wire; nothing here is typed
```

2 USB serial (CAN support board ↔ CAN Bridge)

```
board → computer:   $2A1:03,20,00,0A,1F,40
computer → board:   m2A1,03,20,00,0A;
(cansw_usb text format; the board never decodes)
```

3 RegisterComponent (CAN Bridge → Orion, once)

```
RegisterComponent {
  component: "can-upper", session: "s-8F2"
  channels: [{ id: "INJECTOR.ROCKET/PT_CHANNEL_1",
               type: float, unit: "psi" }, ...]
  commands: [{ type: "ACTUATOR_CMD", params: {
    actuator: enum[ACTUATOR_OX_INJECTOR_VALVE, ...],
    state: enum[ACT_STATE_ON, ACT_STATE_OFF] } }]
}
```

4 PublishTelemetry (CAN Bridge → Orion)

```
opened once when the CAN Bridge starts, then it keeps sending:

TelemetryBatch {
    component: "can-upper", session: "s-8F2",
    seq: 1042, channel: "INJECTOR.ROCKET/PT_CHANNEL_1",
    time_basis: "board-ms", times: [37.590],
    values: [800.0], quality: VALID }
TelemetryBatch { seq: 1043, values: [801.2], ... }
TelemetryBatch { seq: 1044, values: [799.8], ... }

missing seq number = lost data, flagged instead of hidden
```

5 PublishTelemetry (DAQ Component → Orion)

```
TelemetryBatch {
  component: "daq-main", seq: 88120
  channel: "PT-OX-TANK", unit: "psi"
  time_basis: "acquisition-clock"
  times: [12.000 ... 12.024]   (25 samples @ 1 kHz)
  values: [612.4, 612.6, ...]
  quality: VALID
}
```

6 Snapshot + Subscribe (Orion → Launch App)

```
GetSnapshot{["can-upper/INJECTOR.ROCKET/PT_CHANNEL_1"]}
  → { revision: 5512,
      value: 800.0 psi, age: 40 ms, VALID }
SubscribeTelemetry{channels, from_revision: 5512}
  → TelemetryEvent { rev: 5513, value: 801.2, VALID }
  → StatusEvent { component: "can-upper", STALE }
```

7 SubmitCommand / WatchCommand (Launch App ↔ Orion)

```
SubmitCommand {
  request_id: "op2-17", target: "can-upper"
  command: "ACTUATOR_CMD"
  params: { actuator: "ACTUATOR_OX_INJECTOR_VALVE",
            state: "ACT_STATE_ON" }
  expires_in_ms: 2000 }
  → { command_id: "cmd-17", state: ACCEPTED }
WatchCommand → DISPATCHED → CONFIRMED | UNKNOWN
```

8 CommandDispatch (Orion → CAN Bridge → board)

```
CommandChannel(stream CommandStatus) → stream CommandDispatch
CommandDispatch { command_id: "cmd-17",
  command: "ACTUATOR_CMD", params: { ... },
  expires_at: 12:04:31.200 }
CAN Bridge: Parsley encode + USB write
    m0C3,00,00,0C,01;   (illustrative bytes)
ReportCommandResult { "cmd-17", DISPATCHED }
CONFIRMED only when the board reports ON
```

9 RLCS v3 frame (Clientside ↔ laptop, USB)

```
binary, 115200 baud:
  'W' | switches | valve states | currents | ... | 'R'
laptop → RLCS override:
  'W' OVERRIDE OPEN 'R'   (resend < 3 s,
   or the firmware drops it: fail-safe)

Clientside → Towerside over the RLCS radio Link (not via Orion)
```

10 Safety-critical path (RLCS Component → Orion)

```
TelemetryBatch { component: "rlcs-main",
  channel: "QD301 Command", values: ["OPEN"] }
SubmitCommand {
  request_id: "rlcs-sw-0412", target: "can-upper"
  command: "ACTUATOR_CMD", params: {
    actuator: "ACTUATOR_OX_INJECTOR_VALVE",
    state: "ACT_STATE_ON" },
  expires_in_ms: 500 }
```

11 Log record (Orion → Logger, file (#71))

```
header: { format: "orion-log", version: 1,
          rocketcan: "2026.3" }
record: { t_recv: 1783291891.284,
  component: "can-upper",
  channel: "INJECTOR.ROCKET/PT_CHANNEL_1",
  t_src: 37.595, value: 800.0, quality: VALID }
```

12 Direct USB mode (USB debug → Launch App)

```
$2A1:03,20,00,0A,1F,40
  = parsley-ts decode
  + SAME channel model as 6:
{ channel: "usb/INJECTOR.ROCKET/PT_CHANNEL_1",
  value: 800.0, unit: "psi", quality: VALID }
no Orion: no command checks, no Logger
```

13 RF telemetry (rocket radio ↔ ground radios)

```
the same CAN frames as 1, relayed over 920 MHz
ground radio board: RF + CAN + USB debug
+ CAN Bridge "can-telem" on a mission-control computer → Orion
(same frame may also arrive via umbilical → dedup)
```

Heartbeat (every component → Orion)

```
ReportHealth { component: "can-upper",
  session: "s-8F2", status: OK,
  link: "USB up", frames_per_s: 412,
  crc_errors: 0 }
no heartbeat for 2 s → Orion marks STALE
and every app shows it
```

Compare: the same reading in Omnibus today

```
send("CAN/Parsley", {
  "board_type_id": "INJECTOR",
  "board_inst_id": "ROCKET",
  "msg_type": "SENSOR_ANALOG16",
  "msg_metadata": "SENSOR_PT_CHANNEL_1",
  "data": {"time": 37.595, "value": 800} })
no schema · no unit · no version · no quality
every app must know this dict (dashboard/parsers.py)
```

API (sample, gRPC):

```
Component side
  RegisterC(component: string, channels: ChannelInfo[], commands: CommandInfo[]))
    -> (session: string)
  PublishTelemetry(stream TelemetryBatch) → (last_seq_accepted: uint64)
  ReportHealth(stream HealthReport) → ()
  CommandChannel(stream CommandStatus) → stream CommandDispatch
App side
  ListChannels({component_filter: string}) → {channels: ChannelInfo[]}
  GetSnapshot{{channels: string[]}} → {revision: uint64, values: ChannelValue[]}
  SubscribeTelemetry{{channels: string[], from_revision: uint64}} → stream TelemetryEvent
  SubmitCommand{SubmitCommandRequest} → CommandStatus   # ACCEPTED | REJECTED
  WatchCommand{{command_id: string}} → stream CommandStatus
```

Example Orion Telemetry Data Model

Shared Telemetry

```
message TelemetryBatch {
  string component_id = 1;
  string session_id = 2;
  uint64 sequence = 3;
  SourceInfo source = 4;
  double published_at_unix = 5;
  repeated double sample_times = 6;
  repeated NumericSamples samples = 7;
  optional double configured_sample_rate_hz = 8;
  optional double actual_sample_rate_hz = 9;
}
```

Decoded Channel Readings

```
message NumericSamples {
  string channel_id = 1;
  repeated double values = 2;
  string device_id = 3;
}
```

CAN and DAQ keep their own components. Both publish decoded values through this shared model.

Example Orion Source Metadata

Source Fields and Types

```
message SourceInfo {
  string channel = 1;
  string instance_id = 2;
  uint32 format_version = 3;
  SourceType source_type = 4;
}

enum SourceType {
  SOURCE_TYPE_UNSPECIFIED = 0;
  DAQ = 1;
  CAN = 2;
  OMNIBUS_ADAPTER = 3;
  REPLAY = 4;
}
```

DAQ

```
var daqSource *pb.SourceInfo = &pb.SourceInfo{
  SourceType: pb.SourceType_DAQ,
  Channel: "daq-main",
  InstanceId: "daq-main",
  FormatVersion: 1,
}
```

CAN

```
var canSource *pb.SourceInfo = &pb.SourceInfo{
  SourceType: pb.SourceType_CAN,
  Channel:    "rocket-can",
  InstanceId: "can-upper",
  FormatVersion: 1,
}
```

Comparison with Omnibus - DAQ

Omnibus

```
sender.send("DAQ/ljm", {
    "timestamp": 1700000000.025,
    "data": {
        "PT-OX-TANK (psi)": [612.4, 612.6],
        "Voltage (V)": [1.50, 1.51],
    },
    "relative_timestamps": [
        1700000000.000,
        1700000000.001,
    ],
    "sample_rate": 1000,
    "message_format_version": 3,
})
```

Proposed Orion

```
      var batch *pb.TelemetryBatch
      batch = &pb.TelemetryBatch{
        ComponentId: "daq-main", SessionId: "run-42",
        Sequence: 88120, Source: daqSource,
        PublishedAtUnix: 1700000000.026,
        ConfiguredSampleRateHz: proto.Float64(1000),
        ActualSampleRateHz: proto.Float64(1000),
        SampleTimes: []float64{1.7e9, 1700000000.001},
        Samples: []*pb.NumericSamples{
          &pb.NumericSamples{
            DeviceId: "daq-main", ChannelId: "PT-OX-
TANK",
            Values: []float64{612.4, 612.6},
          },
          &pb.NumericSamples{
            DeviceId: "daq-main", ChannelId: "Voltage",
            Values: []float64{1.50, 1.51},
          },
        },
      }
      stream.Send(batch)
```

Comparison with Omnibus - CAN

Omnibus

```
sender.send("CAN/Parsley", {
    "board_type_id": "INJECTOR",
    "board_inst_id": "ROCKET",
    "msg_prio": "LOW",
    "msg_type": "SENSOR_ANALOG32",
    "msg_metadata": "SENSOR_PT_CHANNEL_1",
    "data": {
        "time": 37.595,
        "value": 1310,
    },
    "parsley": "daq/usb//dev/ttyACM0",
    "message_format_version": 2,
})
```

Proposed Orion

```
var batch *pb.TelemetryBatch
batch = &pb.TelemetryBatch{
  ComponentId: "can-main", SessionId: "run-17",
  Sequence: 1, Source: canSource,
  PublishedAtUnix: 1700000000.030,
  SampleTimes: []float64{37.595},
  Samples: []*pb.NumericSamples{
    &pb.NumericSamples{
      DeviceId: "injector-rocket",
      ChannelId: "PT_CHANNEL_1",
      Values: []float64{1310},
    },
  },
}
stream.Send(batch)
```

Example Orion Command Fields

Command Request

```proto
message CommandRequest {
  string request_id = 1;
  string target = 2;
  double expires_at_unix = 3;
  double issued_at_unix = 4;
  string operation = 5;
  map<string, string> params = 6;
}
```

Receipt and Status

```proto
message CommandReceipt {
  string command_id = 1;
  CommandStatus status = 2;
}

enum CommandStatus {
  COMMAND_STATUS_UNSPECIFIED = 0;
  ACCEPTED = 1;
  DISPATCHED = 2;
  CONFIRMED = 3;
  FAILED = 4;
  EXPIRED = 5;
  UNKNOWN = 6;
}
```

The app sends named parameters as strings. The catalog defines their types and allowed values; the server validates before routing.

Comparison with Omnibus - Commands

Omnibus

```
sender.send("CAN/Commands", {
    "parsley": "EGSE-UPPER/usb//dev/ttyACM0",
    "data": {
        "time": 1700000000.0,
        "can_msg": {
            "msg_prio": "HIGHEST",
            "msg_type": "ACTUATOR_CMD",
            "board_type_id": "DAQ",
            "board_inst_id": "GROUND",
            "time": 0,
            "msg_metadata":
"ACTUATOR_OX_INJECTOR_VALVE",
            "cmd_state": "ACT_STATE_ON",
        },
    },
})
```

Proposed Orion

```
var request *pb.CommandRequest
request = &pb.CommandRequest{
  RequestId: "op2-17",
  Target: "ox-injector-valve",
  Operation: "set-state",
  Params: map[string]string{"state": "ON"},
  IssuedAtUnix: 1700000000.0,
  ExpiresAtUnix: 1700000002.0,
}
commandClient.SubmitCommand(ctx, request)
// Example reply: cmd-17, ACCEPTED
```

How the Component Maps Commands

The app requests an action on a registered target. Orion routes it to the configured owner.

Orion Request	Inside the CAN Component
Target: `ox-injector-valve`	Lookup: `ground-daq` , actuator binding
Operation: `set-state`	`msg_type = ACTUATOR_CMD`
Parameter: `state = ON`	`cmd_state = ACT_STATE_ON`
Target's private setup	`DAQ / GROUND` , `ACTUATOR_OX_INJECTOR_VALVE`

Unary vs. Streaming

Unary

```
var receipt *pb.CommandReceipt
receipt, _ = client.SubmitCommand(ctx, request)
return receipt
```

Ask to open a valve. The reply confirms acceptance, not movement.

Stream

```
var stream pb.Telemetry_PublishTelemetryClient
stream, _ = client.PublishTelemetry(ctx)
for {
  // batch = { ... }
  stream.Send(batch)
}
stream.CloseAndRecv()
```

Telemetry data are sent continuously through a stream.

Sample APIs

Sources

```
service Components {
  rpc RegisterComponent(RegisterRequest)
    returns (RegisterReply);
  rpc ReportHealth(HealthReport)
    returns (Ack);
}
```

Discovery

```
service Discovery {
  rpc ListDevices(DeviceFilter)
    returns (DeviceCatalog);
  rpc ListCommands(CommandFilter)
    returns (CommandCatalog);
  rpc ListChannels(ChannelFilter)
    returns (ChannelCatalog);
  rpc WatchComponentStatus(StatusFilter)
    returns (stream ComponentStatus);
}
```

Telemetry

```
service Telemetry {
  rpc PublishTelemetry(stream TelemetryBatch)
    returns (PublishResult);
  rpc SubscribeTelemetry(SubscriptionRequest)
    returns (stream TelemetryEvent);
}
```

Commands

```
service Commands {
  rpc SubmitCommand(CommandRequest)
    returns (CommandReceipt);
  rpc WatchCommand(WatchCommandRequest)
    returns (stream CommandReceipt);
}
```

SECTION 08

Testing

08

WATERLOO ROCKETRY

Testing

TL;DR: Everything gets unit tests.

- Keep intended behaviour well-defined for future maintainers, so we can fix bugs and add functionality without breaking existing workflows.
- Aim for **85% test coverage**, with explicit tests for invalid input, disconnects and failure handling.
- Add integration tests across the stack, including the server, sources, sinks, and applications.
  - Use Fake Source and replay for repeatable tests without physical hardware.
- Automate end-to-end testing where practical; the tooling and scope are still to be determined.
  - Do manual testing and validation before accepting operational changes.
- Well-maintained dependencies shall be used, and their versions pinned, so builds and test results are reproducible.

SECTION 09

Project Management

09

WATERLOO ROCKETRY

Timelines, and Reviews

`2026-2027-software-issues#51` contains the timeline and milestones for the Orion project.

Duration: About 9 Months, from Early October 2026 to Mid July 2027.

- Design and Specifications: October - Mid November;
- Prototype Work through Late November.
- Adapters, Deployment and Migration Preparation: Start in November
  - Continue Alongside Implementation.
- Native Implementation: Early January - Late June; CDR: Late January - Early February.
- Full Integration and Readiness Checks: Late February - Mid July.

SECTION 10

Action Items

10

WATERLOO ROCKETRY

Action Items

1. Review the issues in `2026-2027-software-issues#51` and its children.
2. Start working on the specification, including the definition of the communication protocol, data model, API, and configuration format.
3. Start working on the prototype, including the server, source and sink modules.
4. Start building LJM library in Go.
5. Plan for implementation of the Omnibus adapter which should allow Omnibus sources and sinks to work with Orion server.